# Product Recommendation from Transaction History: Project Report

Date: November 2026 | Status: complete

## 1. Goal

Build a model that reads a bank customer's transaction history and predicts which of five products they are likely to take up: travel card, personal loan, investments, insurance or cashback plus. The best-scoring product is the one we offer.

## 2. What was done

- Created a practice dataset of 20,000 customers and 4.25 million transactions, planned with HLD/LLD design.
- Adapted an open AI language model, Qwen3-1.7B, to this task, training it twice on Kaggle's free GPUs (about 4 and 6.5 hours).
- Tested it on 5,069 customers from later months than the training data, against four other models.

## 3. Our approach

Most banks summarise each customer into totals, such as "spend in the last 90 days", and score those. Totals lose the order of events: a pay rise followed by investing can look the same as late fees followed by a loan. Our model reads the individual transactions in order, combines them with the customer's profile, and learns from both at the same time. Figure 1 shows the steps.

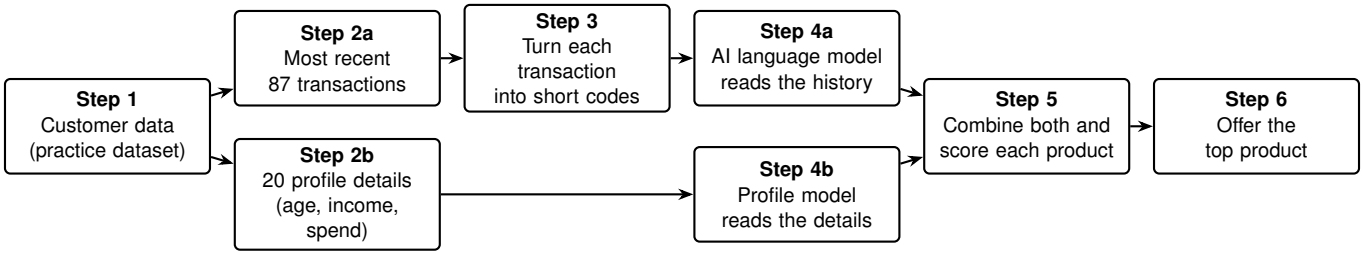


**Figure 1.** How a recommendation is made. In training, steps 4a, 4b and 5 learn together from whether the customer took the offer.

## 4. Results

Each model is scored on how well it ranks customers who took the product above those who did not: 0.5 means random guessing and 1.0 means perfect (this measure is called AUC). Our model scored 0.736, which is 3.8% below the standard model (0.765).

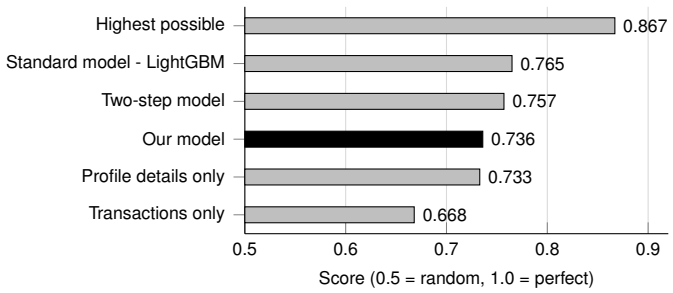


**Figure 2.** Test scores; our model is the black bar.

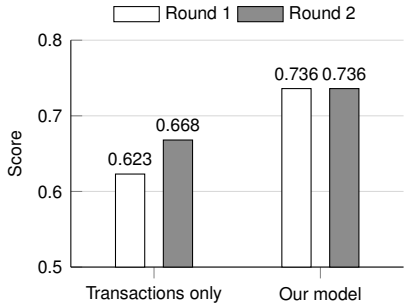


**Figure 3.** Score in round 1 and round 2.

## 5. Problems found and how we solved them

| Problem | What we did | Result |
|---|---|---|
| No real bank data available | Built a practice dataset that also records each customer's true chance of taking each product | We know the highest score possible |
| Transactions written as text are long, and the model reads amounts poorly | Replaced each transaction with short codes for amount range, date and shop name | About 12 codes per transaction |
| The model is too large to train fully on free GPUs | Trained only small add-on parts (about 1% of the model), with memory-saving settings | Uses 7 GB of a 16 GB GPU |
| Risk of testing on data the model had already seen | Trained on months 1–8, tuned on month 9, tested on months 10–12 | A fair test on future customers |
| Round 1 saw only the last 37 transactions | Round 2 doubled how much history the model reads | Transactions-only score rose from 0.623 to 0.668 |
| Round 1 ranked the five products worse than chance | Round 2 gave each product its own score | Product ranking now better than chance |

**Table 1.** Problems found during the project and how each was handled.

## 6. Conclusion

The system works from start to finish on free hardware, and transaction history alone predicts take-up well above random. It does not yet beat the standard model, mainly because the training set is small (13,000 customers) and the model has not first learned spending patterns.

## 7. Suggested steps to improve the results

1. **Teach the model spending patterns first.** Train it to predict the next transaction, then to recommend.
2. **Use more customers and train longer.** Use 50,000+ training customers and three passes instead of one.
3. **Use a larger test set.** About 55,000 test customers are needed to measure small improvements reliably.
4. **Change one thing at a time,** so each training round shows clearly what helped.
5. **Get more computing time.** Round 2 already used 6.4 of Kaggle's 12 free hours; longer training needs a paid GPU.
6. **Test on real, anonymised bank data** when available, to confirm the results hold outside the practice dataset.